

FILTERING

Apa Itu Filter?

Filter dalam pengolahan citra digital adalah operasi yang memodifikasi nilai intensitas piksel berdasarkan piksel-piksel di sekitarnya. Tujuannya bisa bermacam-macam — mulai dari memperhalus citra (smoothing), menajamkan detail (sharpening), hingga menonjolkan tepi (edge detection).

Secara umum, setiap filter bekerja dengan prinsip:



No description has been provided for this image

Hasil gambar yang difilter (Y) dari gambar asal (X) dan filter dengan kernel (H)

Konvolusi

Konvolusi merupakan operasi dasar dalam pengolahan citra digital yang digunakan untuk menerapkan berbagai jenis filter seperti blur, sharpen, maupun edge detection. Secara sederhana, konvolusi adalah proses menggabungkan dua fungsi — yaitu citra dan filter — untuk menghasilkan citra baru yang telah dimodifikasi.

Pada rumus dasar sebelumnya, konvolusi dari dua fungsi H dan X didefinisikan sebagai:



No description has been provided for this image

setiap titik citra pada posisi (Tx, Ty) , nilai piksel hasil konvolusi dihitung dengan mengalikan setiap elemen filter $H(x, y)$ dengan nilai piksel citra X di posisi yang bersesuaian, kemudian menjumlahkan seluruh hasil perkalian tersebut.

Dengan kata lain, konvolusi bekerja dengan cara menggeser filter ke seluruh bagian citra, lalu menghitung hasil perkalian elemen demi elemen (disebut dot product) antara nilai filter dan area citra yang diliputi oleh filter tersebut.

$H(x, y)$ adalah nilai dari elemen filter atau kernel pada posisi (x, y) .

$X(Tx-x, Ty-y)$ adalah nilai piksel citra yang dikalikan dengan elemen filter pada posisi yang sesuai.

Simbol $\sum_y \sum_x$ menunjukkan bahwa semua hasil perkalian dijumlahkan untuk menghasilkan satu nilai keluaran di titik (Tx, Ty)

Bagian $(Tx-x, Ty-y)$ menunjukkan bahwa filter dibalik secara horizontal dan vertikal sebelum digunakan. Inilah yang membedakan konvolusi murni dari korelasi. Jika filter tidak dibalik, operasi yang terjadi sebenarnya adalah korelasi, bukan konvolusi.

FFT (Fast Fourier Transform)

FFT (Fast Fourier Transform) pada gambar digital adalah proses untuk mengubah representasi gambar dari domain spasial (ruang) menjadi domain frekuensi.

Kalau di domain spasial kita melihat gambar sebagai kumpulan nilai intensitas piksel (misalnya terang–gelap di tiap titik), maka di domain frekuensi kita melihat gambar sebagai kombinasi gelombang sinusoidal dengan berbagai frekuensi dan amplitudo yang, jika digabungkan, akan membentuk gambar tersebut.

FFT pada Gambar: Fungsi dan Cara Menganalisis

1. Konsep Dasar

Gambar digital dapat dipandang sebagai fungsi dua dimensi $f(x, y)$ di domain spasial. **Fast Fourier Transform** mengubahnya menjadi representasi domain frekuensi $F(u, v)$. Frekuensi rendah merepresentasikan perubahan yang lambat atau area halus, sedangkan frekuensi tinggi merepresentasikan tepi dan detail halus.

2. Fungsi Utama FFT dalam Pengolahan Citra

- **Transformasi ke domain frekuensi** sehingga kita dapat melihat sebaran energi berdasarkan frekuensi.
- **Analisis struktur dan pola** seperti halus atau kasar, acak atau berulang, serta arah dominan suatu tekstur.

3. Cara Membaca Spektrum FFT

Bagian Spektrum	Makna	Ciri Visual
Pusat	Frekuensi rendah, menyimpan informasi global dan gradien halus	Area terang di tengah setelah <code>fftshift</code>
Pinggiran	Frekuensi tinggi, menyimpan tepi dan detail halus	Titik atau pola terang di tepi kanvas spektrum
Arah pola	Orientasi dominan tekstur atau garis pada gambar	Garis terang yang saling berhadapan sesuai orientasi
Simetri	Citra real menghasilkan spektrum yang simetris	Kiri dan kanan serta atas dan bawah tampak sebagai cerminan

4. Contoh Analisis Kasus Visual

- **Gambar halus** seperti langit atau wajah: pusat sangat terang, pinggir cenderung gelap karena dominasi frekuensi rendah.
- **Gambar penuh tepi** seperti tulisan atau rambut: banyak komponen terang di pinggir yang menandakan frekuensi tinggi berlimpah.
- **Gambar bertekstur atau periodik** seperti kain bermotif: muncul pola teratur di spektrum yang menandakan periodisitas.

5. Fungsi Analisis FFT untuk Tugas Praktis

Tujuan Analisis	Penjelasan	Contoh Aplikasi
Identifikasi frekuensi dominan	Mengukur proporsi energi rendah dibanding energi tinggi	Klasifikasi tekstur atau penilaian kualitas gambar
Analisis orientasi pola	Menentukan arah perubahan dominan pada struktur	Deteksi arah serat, retakan, atau pola kisi
Kompresi serta penyaringan	Mengurangi frekuensi tinggi yang kurang penting secara visual	Kompresi citra dan denoising
Analisis ketajaman	Frekuensi tinggi menunjukkan detail serta ketajaman	Penilaian fokus kamera dan mikroskop

6. Tips Membaca Spektrum dengan Baik

- Gunakan `fftshift` agar komponen nol berada di tengah sehingga mudah dianalisis.
- Tampilkan magnitude dalam skala logaritmik menggunakan `$\log(1 + |F|)$` agar rentang dinamis terlihat jelas.
- Amati pola simetri dan arah garis terang untuk memahami orientasi tekstur.
- Bandingkan beberapa gambar dengan jenis konten yang berbeda untuk melihat perbedaan sidik jari frekuensi.
- Verifikasi pemahaman dengan melakukan inverse FFT setelah menahan atau menghilangkan rentang frekuensi tertentu.

7. Ringkasan Intuitif

FFT adalah kacamata frekuensi yang menampilkan ritme perubahan pada gambar. Pusat spektrum menyimpan bentuk global sementara pinggiran menyimpan tepi dan detail. Dengan membaca pola terang, arah, dan sebaran energi, kita dapat menilai karakter gambar secara kuantitatif dan kualitatif.

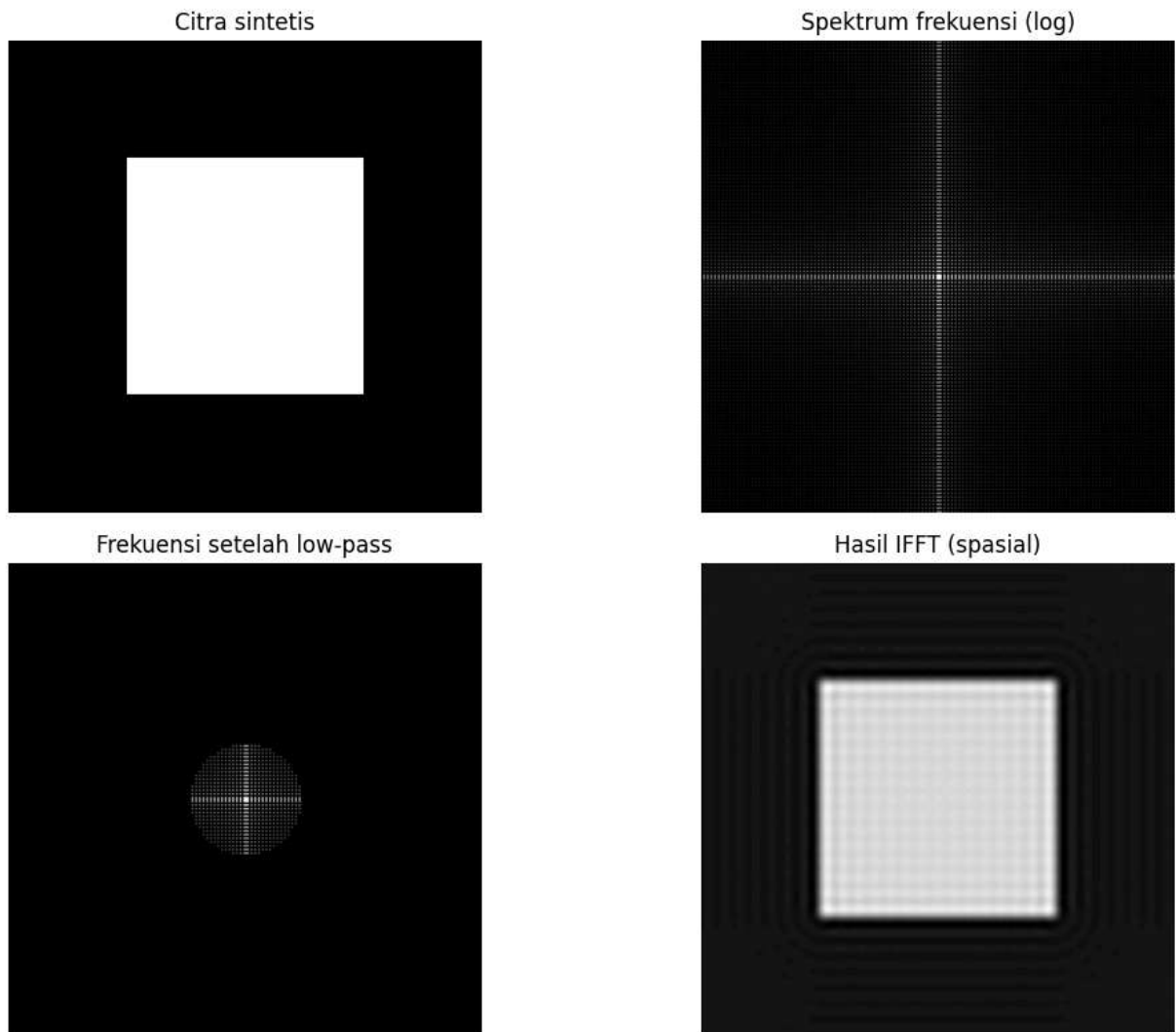
```
In [9]: import numpy as np
import matplotlib.pyplot as plt
from numpy.fft import fft2, fftshift, ifft2
```

```
# Buat citra sintetis: kotak putih di tengah, ukuran 256x256
N = 256
image = np.zeros((N, N), dtype=float)
image[64:192, 64:192] = 1.0 # kotak

# FFT 2D
F = fft2(image)
F_shift = fftshift(F)
magnitude = np.log1p(np.abs(F_shift))

# Contoh Low-pass sederhana di domain frekuensi
R = 30 # radius cutoff
yy, xx = np.ogrid[:N, :N]
cy, cx = N//2, N//2
mask = (yy - cy)**2 + (xx - cx)**2 <= R**2
F_lp = F_shift * mask
lp_image = np.real(iff2(fftshift(F_lp)))

# Visualisasi
plt.figure(figsize=(12,8))
plt.subplot(2,2,1); plt.imshow(image, cmap='gray'); plt.title("Citra sintetis"); pl
plt.subplot(2,2,2); plt.imshow(magnitude, cmap='gray'); plt.title("Spektrum frekuen
plt.subplot(2,2,3); plt.imshow(np.log1p(np.abs(F_lp)), cmap='gray'); plt.title("Fre
plt.subplot(2,2,4); plt.imshow(lp_image, cmap='gray'); plt.title("Hasil IFFT (spasi
plt.tight_layout(); plt.show()
```



```
In [1]: import cv2
import matplotlib.pyplot as plt

# 1) Load BGR, ubah ke RGB biar konsisten dengan Matplotlib
bgr = cv2.imread("91.jpg", cv2.IMREAD_UNCHANGED)

if bgr is None:
    raise FileNotFoundError("File 'lena.png' tidak ditemukan di direktori kerja saa

# Jika ada alfa, pisahkan dulu
if bgr.shape[-1] == 4:
    bgr, alpha = bgr[..., :3], bgr[..., 3]

rgb = cv2.cvtColor(bgr, cv2.COLOR_BGR2RGB)
rgb_f = rgb.astype("float32")/255.0

# 2) Grayscale manual supaya konsisten dengan opsi A
R, G, B = rgb_f[..., 0], rgb_f[..., 1], rgb_f[..., 2]
gray = 0.299*R + 0.587*G + 0.114*B

# 3) Visualisasi
plt.figure(figsize=(10,4))
plt.subplot(1,2,1); plt.imshow(rgb_f); plt.title("Lena RGB"); plt.axis("off")
```

```
plt.subplot(1,2,2); plt.imshow(gray, cmap="gray"); plt.title("Lena Grayscale"); plt.
plt.tight_layout(); plt.show()
```

Lena RGB



Lena Grayscale



```
In [2]: import numpy as np
        from numpy.fft import fft2, fftshift

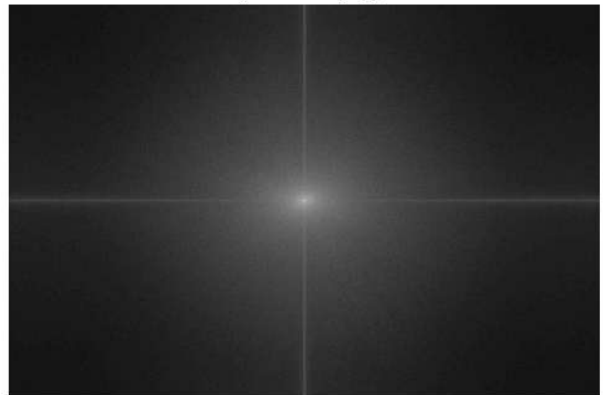
        F = fft2(gray)                # FFT 2D
        F_shift = fftshift(F)         # pusatkan DC di tengah
        spec = np.log1p(np.abs(F_shift)) # magnitude dalam log scale

        plt.figure(figsize=(10,4))
        plt.subplot(1,2,1); plt.imshow(gray, cmap="gray"); plt.title("Grayscale"); plt.axis
        plt.subplot(1,2,2); plt.imshow(spec, cmap="gray"); plt.title("Spektrum (log)"); plt
        plt.tight_layout(); plt.show()
```

Grayscale



Spektrum (log)



```
In [4]: import cv2
        import matplotlib.pyplot as plt

        # 1) Load BGR, ubah ke RGB biar konsisten dengan Matplotlib
        bgr = cv2.imread("01.jpg", cv2.IMREAD_UNCHANGED)

        if bgr is None:
            raise FileNotFoundError("File 'lena.png' tidak ditemukan di direktori kerja saa

        # Jika ada alfa, pisahkan dulu
        if bgr.shape[-1] == 4:
            bgr, alpha = bgr[..., :3], bgr[..., 3]
```



```

rgb = cv2.cvtColor(bgr, cv2.COLOR_BGR2RGB)
rgb_f = rgb.astype("float32")/255.0

# 2) Grayscale manual supaya konsisten dengan opsi A
R, G, B = rgb_f[...,0], rgb_f[...,1], rgb_f[...,2]
gray = 0.299*R + 0.587*G + 0.114*B

# 3) Visualisasi
plt.figure(figsize=(10,4))
plt.subplot(1,2,1); plt.imshow(rgb_f); plt.title("Lena RGB"); plt.axis("off")
plt.subplot(1,2,2); plt.imshow(gray, cmap="gray"); plt.title("Lena Grayscale"); plt
plt.tight_layout(); plt.show()

```

Lena RGB



Lena Grayscale



```

In [5]: import numpy as np
        from numpy.fft import fft2, fftshift

        F = fft2(gray)                                # FFT 2D
        F_shift = fftshift(F)                         # pusatkan DC di tengah
        spec = np.log1p(np.abs(F_shift))               # magnitude dalam log scale

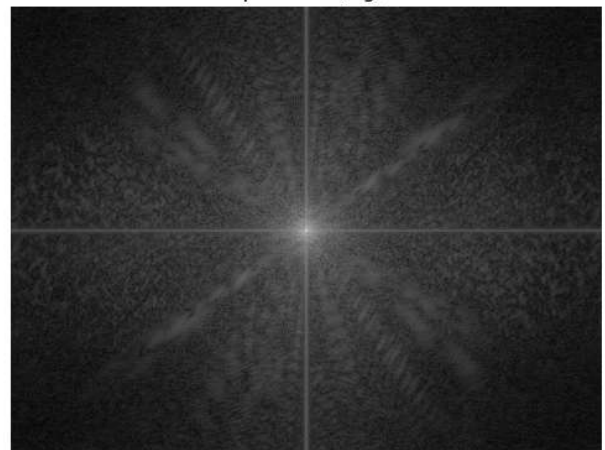
        plt.figure(figsize=(10,4))
        plt.subplot(1,2,1); plt.imshow(gray, cmap="gray"); plt.title("Grayscale"); plt.axis
        plt.subplot(1,2,2); plt.imshow(spec, cmap="gray"); plt.title("Spektrum (log)"); plt
        plt.tight_layout(); plt.show()

```

Grayscale



Spektrum (log)



```

In [6]: import numpy as np

# Matriks sesuai gambar
H = np.array([[1, 1, 1],
               [1, 4, 1],
               [1, 1, 1]], dtype=np.int64) # kernel 3x3

X = np.array([[1, 0, 0, 0],
               [1, 1, 1, 0],
               [1, 1, 1, 0],
               [1, 0, 0, 0]], dtype=np.int64) # citra 4x4

# Konvolusi 2D manual: definisi matematis (membalik kernel)
def conv2_valid(img, kernel):
    kh, kw = kernel.shape
    Hh, Hw = img.shape
    kflip = kernel[::-1, ::-1]
    out = np.zeros((Hh - kh + 1, Hw - kw + 1), dtype=np.int64)
    for i in range(out.shape[0]):
        for j in range(out.shape[1]):
            window = img[i:i+kh, j:j+kw]
            out[i, j] = np.sum(window * kflip)
    return out

def conv2_same(img, kernel, pad_mode="constant"):
    kh, kw = kernel.shape
    ph, pw = kh // 2, kw // 2
    imgp = np.pad(img, ((ph, ph), (pw, pw)), mode=pad_mode)
    return conv2_valid(imgp, kernel)

def conv2_full(img, kernel):
    kh, kw = kernel.shape
    imgp = np.pad(img, ((kh-1, kh-1), (kw-1, kw-1)), mode="constant")
    return conv2_valid(imgp, kernel)

# Hitung tiga mode ukuran keluaran
Y_valid = conv2_valid(X, H)
Y_same = conv2_same(X, H) # zero padding
Y_full = conv2_full(X, H)

print("Input X:\n", X)
print("\nKernel H:\n", H)

print("\nOutput VALID (2x2):\n", Y_valid)
print("\nOutput SAME (4x4, zero padding):\n", Y_same)
print("\nOutput FULL (6x6):\n", Y_full)

# Jika ingin membandingkan dengan korelasi 2D
def corr2_valid(img, kernel):
    kh, kw = kernel.shape
    Hh, Hw = img.shape
    out = np.zeros((Hh - kh + 1, Hw - kw + 1), dtype=np.int64)
    for i in range(out.shape[0]):
        for j in range(out.shape[1]):
            window = img[i:i+kh, j:j+kw]

```



```

        out[i, j] = np.sum(window * kernel)
    return out

C_valid = corr2_valid(X, H)
print("\nKorelasi VALID:\n", C_valid, "\n(untuk kernel simetris seperti H nilainya

```

Input X:

```

[[1 0 0 0]
 [1 1 1 0]
 [1 1 1 0]
 [1 0 0 0]]

```

Kernel H:

```

[[1 1 1]
 [1 4 1]
 [1 1 1]]

```

Output VALID (2x2):

```

[[10  7]
 [10  7]]

```

Output SAME (4x4, zero padding):

```

[[ 6  4  2  1]
 [ 8 10  7  2]
 [ 8 10  7  2]
 [ 6  4  2  1]]

```

Output FULL (6x6):

```

[[ 1  1  1  0  0  0]
 [ 2  6  4  2  1  0]
 [ 3  8 10  7  2  0]
 [ 3  8 10  7  2  0]
 [ 2  6  4  2  1  0]
 [ 1  1  1  0  0  0]]

```

Korelasi VALID:

```

[[10  7]
 [10  7]]

```

(untuk kernel simetris seperti H nilainya sama dengan konvolusi)

```

In [ ]: import numpy as np
        from PIL import Image
        import matplotlib.pyplot as plt

        # ==== 1) Baca dan ubah ke grayscale 0..1 ====
        img_rgb = Image.open("01.jpg").convert("RGB") # convert RGB to GRAY
        arr = np.asarray(img_rgb).astype(np.float32) / 255.0
        R, G, B = arr[...,0], arr[...,1], arr[...,2]
        gray = 0.299*R + 0.587*G + 0.114*B # Luminance BT.601

        print("Ukuran grayscale:", gray.shape)

        # ==== 2) Kernel H sesuai contoh ====
        H = np.array([[1, 1, 1],
                      [1, 4, 1],
                      [1, 1, 1]], dtype=np.float32)

```

```

# ==== 3) Fungsi konvolusi 2D manual (membalik kernel) ====
def conv2_valid(img, kernel):
    kh, kw = kernel.shape
    ih, iw = img.shape
    kflip = kernel[::-1, ::-1]
    oh, ow = ih - kh + 1, iw - kw + 1
    out = np.zeros((oh, ow), dtype=np.float32)
    for i in range(oh):
        for j in range(ow):
            window = img[i:i+kh, j:j+kw]
            out[i, j] = np.sum(window * kflip)
    return out

def conv2_same(img, kernel, pad_mode="constant"):
    kh, kw = kernel.shape
    ph, pw = kh // 2, kw // 2
    if pad_mode == "replicate":
        imgp = np.pad(img, ((ph, ph), (pw, pw)), mode="edge")
    elif pad_mode == "reflect":
        imgp = np.pad(img, ((ph, ph), (pw, pw)), mode="reflect")
    else:
        imgp = np.pad(img, ((ph, ph), (pw, pw)), mode="constant", constant_values=0)
    return conv2_valid(imgp, kernel)

def conv2_full(img, kernel):
    kh, kw = kernel.shape
    imgp = np.pad(img, ((kh-1, kh-1), (kw-1, kw-1)), mode="constant")
    return conv2_valid(imgp, kernel)

# ==== 4) Hitung keluaran ====
Y_valid = conv2_valid(gray, H) # Lebih kecil
Y_same0 = conv2_same(gray, H, "constant") # ukuran sama, padding nol
Y_samerep = conv2_same(gray, H, "replicate") # ukuran sama, tepi direplikasi
Y_sameref = conv2_same(gray, H, "reflect") # ukuran sama, refleksi
Y_full = conv2_full(gray, H) # Lebih besar

print("valid:", Y_valid.shape, " same:", Y_same0.shape, " full:", Y_full.shape)

# ==== 5) Normalisasi opsional untuk ditampilkan ====
def norm01(x):
    x = x.astype(np.float32)
    m1, m2 = np.percentile(x, 1), np.percentile(x, 99)
    x = np.clip((x - m1) / (m2 - m1 + 1e-8), 0, 1)
    return x

# ==== 6) Visualisasi ====
plt.figure(figsize=(14,8))

plt.subplot(2,3,1); plt.imshow(gray, cmap="gray"); plt.title("Lena Grayscale"); plt
plt.subplot(2,3,2); plt.imshow(norm01(Y_same0), cmap="gray"); plt.title("Same, zero
plt.subplot(2,3,3); plt.imshow(norm01(Y_samerep), cmap="gray"); plt.title("Same, re

plt.subplot(2,3,5); plt.imshow(norm01(Y_sameref), cmap="gray"); plt.title("Same, re
plt.subplot(2,3,4); plt.imshow(norm01(Y_valid), cmap="gray"); plt.title("Valid"); p
plt.subplot(2,3,6); plt.imshow(norm01(Y_full), cmap="gray"); plt.title("Full"); plt

```

```
plt.tight_layout()
plt.show()
```

Ukuran grayscale: (639, 843)

```
In [ ]: import numpy as np
        from PIL import Image
        import matplotlib.pyplot as plt

        # ==== 1) Baca dan ubah ke grayscale 0..1 ====
        img_rgb = Image.open("01.png").convert("RGB")
        arr = np.asarray(img_rgb).astype(np.float32) / 255.0
        R, G, B = arr[...,0], arr[...,1], arr[...,2]
        gray = 0.299*R + 0.587*G + 0.114*B    # Luminance BT.601

        print("Ukuran grayscale:", gray.shape)

        # ==== 2) Kernel H sesuai contoh ====
        H = np.array([[1, 1, 1],
                      [1, 4, 1],
                      [1, 1, 1]], dtype=np.float32)

        H = 1/12 * H
        # ==== 3) Fungsi konvolusi 2D manual (membalik kernel) ====
        def conv2_valid(img, kernel):
            kh, kw = kernel.shape
            ih, iw = img.shape
            kflip = kernel[::-1, ::-1]
            oh, ow = ih - kh + 1, iw - kw + 1
            out = np.zeros((oh, ow), dtype=np.float32)
            for i in range(oh):
                for j in range(ow):
                    window = img[i:i+kh, j:j+kw]
                    out[i, j] = np.sum(window * kflip)
            return out

        def conv2_same(img, kernel, pad_mode="constant"):
            kh, kw = kernel.shape
            ph, pw = kh // 2, kw // 2
            if pad_mode == "replicate":
                imgp = np.pad(img, ((ph, ph), (pw, pw)), mode="edge")
            elif pad_mode == "reflect":
                imgp = np.pad(img, ((ph, ph), (pw, pw)), mode="reflect")
            else:
                imgp = np.pad(img, ((ph, ph), (pw, pw)), mode="constant", constant_values=0)
            return conv2_valid(imgp, kernel)

        def conv2_full(img, kernel):
            kh, kw = kernel.shape
            imgp = np.pad(img, ((kh-1, kh-1), (kw-1, kw-1)), mode="constant")
            return conv2_valid(imgp, kernel)

        # ==== 4) Hitung keluaran ====
        Y_valid = conv2_valid(gray, H)           # Lebih kecil
        Y_same0 = conv2_same(gray, H, "constant") # ukuran sama, padding nol
```

```

Y_samerep = conv2_same(gray, H, "replicate") # ukuran sama, tepi direplikasi
Y_sameref = conv2_same(gray, H, "reflect")   # ukuran sama, refleksi
Y_full    = conv2_full(gray, H)              # lebih besar

print("valid:", Y_valid.shape, " same:", Y_same0.shape, " full:", Y_full.shape)

# ==== 5) Normalisasi opsional untuk ditampilkan ====
def norm01(x):
    x = x.astype(np.float32)
    m1, m2 = np.percentile(x, 1), np.percentile(x, 99)
    x = np.clip((x - m1) / (m2 - m1 + 1e-8), 0, 1)
    return x

# ==== 6) Visualisasi ====
plt.figure(figsize=(14,8))

plt.subplot(2,3,1); plt.imshow(gray, cmap="gray"); plt.title("Lena Grayscale"); plt
plt.subplot(2,3,2); plt.imshow(norm01(Y_same0), cmap="gray"); plt.title("Same, zero
plt.subplot(2,3,3); plt.imshow(norm01(Y_samerep), cmap="gray"); plt.title("Same, re

plt.subplot(2,3,5); plt.imshow(norm01(Y_sameref), cmap="gray"); plt.title("Same, re
plt.subplot(2,3,4); plt.imshow(norm01(Y_valid), cmap="gray"); plt.title("Valid"); p
plt.subplot(2,3,6); plt.imshow(norm01(Y_full), cmap="gray"); plt.title("Full"); plt

plt.tight_layout()
plt.show()

```

```

In [ ]: import numpy as np
        from PIL import Image
        import matplotlib.pyplot as plt

# ==== 1) Baca dan ubah ke grayscale 0..1 ====
img_rgb = Image.open("01.png").convert("RGB")
arr = np.asarray(img_rgb).astype(np.float32) / 255.0
R, G, B = arr[...,0], arr[...,1], arr[...,2]
gray = 0.299*R + 0.587*G + 0.114*B # Luminance BT.601

print("Ukuran grayscale:", gray.shape)

# ==== 2) Kernel H sesuai contoh ====
H = np.array([[ -1, 0, 1],
               [ -1, 0, 1],
               [ -3, 0, 3]], dtype=np.float32)

# KERNEL HPF =

# ==== 3) Fungsi konvolusi 2D manual (membalik kernel) ====
def conv2_valid(img, kernel):
    kh, kw = kernel.shape
    ih, iw = img.shape
    kflip = kernel[::-1, ::-1]
    oh, ow = ih - kh + 1, iw - kw + 1
    out = np.zeros((oh, ow), dtype=np.float32)
    for i in range(oh):
        for j in range(ow):
            window = img[i:i+kh, j:j+kw]

```

```

        out[i, j] = np.sum(window * kflip)
    return out

def conv2_same(img, kernel, pad_mode="constant"):
    kh, kw = kernel.shape
    ph, pw = kh // 2, kw // 2
    if pad_mode == "replicate":
        imgp = np.pad(img, ((ph, ph), (pw, pw)), mode="edge")
    elif pad_mode == "reflect":
        imgp = np.pad(img, ((ph, ph), (pw, pw)), mode="reflect")
    else:
        imgp = np.pad(img, ((ph, ph), (pw, pw)), mode="constant", constant_values=0)
    return conv2_valid(imgp, kernel)

def conv2_full(img, kernel):
    kh, kw = kernel.shape
    imgp = np.pad(img, ((kh-1, kh-1), (kw-1, kw-1)), mode="constant")
    return conv2_valid(imgp, kernel)

# ==== 4) Hitung keluaran ====
Y_valid = conv2_valid(gray, H) # Lebih kecil
Y_same0 = conv2_same(gray, H, "constant") # ukuran sama, padding nol
Y_samerep = conv2_same(gray, H, "replicate") # ukuran sama, tepi direplikasi
Y_sameref = conv2_same(gray, H, "reflect") # ukuran sama, refleksi
Y_full = conv2_full(gray, H) # Lebih besar

print("valid:", Y_valid.shape, " same:", Y_same0.shape, " full:", Y_full.shape)

# ==== 5) Normalisasi opsional untuk ditampilkan ====
def norm01(x):
    x = x.astype(np.float32)
    m1, m2 = np.percentile(x, 1), np.percentile(x, 99)
    x = np.clip((x - m1) / (m2 - m1 + 1e-8), 0, 1)
    return x

# ==== 6) Visualisasi ====
plt.figure(figsize=(14,8))

plt.subplot(2,3,1); plt.imshow(gray, cmap="gray"); plt.title("Lena Grayscale"); plt
plt.subplot(2,3,2); plt.imshow(norm01(Y_same0), cmap="gray"); plt.title("Same, zero
plt.subplot(2,3,3); plt.imshow(norm01(Y_samerep), cmap="gray"); plt.title("Same, re

plt.subplot(2,3,5); plt.imshow(norm01(Y_sameref), cmap="gray"); plt.title("Same, re
plt.subplot(2,3,4); plt.imshow(norm01(Y_valid), cmap="gray"); plt.title("Valid"); p
plt.subplot(2,3,6); plt.imshow(norm01(Y_full), cmap="gray"); plt.title("Full"); plt

plt.tight_layout()
plt.show()

```

```

In [ ]: import numpy as np
        from PIL import Image
        import matplotlib.pyplot as plt

        # -----
        # 1) Load & grayscale 0..1
        # -----

```

```

img_rgb = Image.open("91.png").convert("RGB")
arr = np.asarray(img_rgb).astype(np.float32) / 255.0
R, G, B = arr[...,0], arr[...,1], arr[...,2]
gray = 0.299*R + 0.587*G + 0.114*B # Luminance BT.601

# -----
# 2) Tambahkan noise Gaussian
# -----
sigma = 0.1 # atur besaran noise (0.02 s.d. 0.1 umumnya cukup terlihat)
rng = np.random.default_rng(42)
noisy = np.clip(gray + rng.normal(0.0, sigma, size=gray.shape), 0.0, 1.0)

# -----
# 3) Kernel H dan conv2_same
# -----
H = np.array([[1, 1, 1],
               [1, 4, 1],
               [1, 1, 1]], dtype=np.float32)

H_norm = H / H.sum() # normalisasi agar intensitas terjaga (jumlah=1) 1/12 * jumlah

def conv2_valid(img, kernel):
    kh, kw = kernel.shape
    ih, iw = img.shape
    kflip = kernel[::-1, ::-1]
    oh, ow = ih - kh + 1, iw - kw + 1
    out = np.zeros((oh, ow), dtype=np.float32)
    for i in range(oh):
        for j in range(ow):
            window = img[i:i+kh, j:j+kw]
            out[i, j] = np.sum(window * kflip)
    return out

def conv2_same(img, kernel, pad_mode="reflect"):
    kh, kw = kernel.shape
    ph, pw = kh // 2, kw // 2
    if pad_mode == "replicate":
        imgp = np.pad(img, ((ph, ph), (pw, pw)), mode="edge")
    elif pad_mode == "constant":
        imgp = np.pad(img, ((ph, ph), (pw, pw)), mode="constant", constant_values=0)
    else: # default: reflect (recommended for tepi)
        imgp = np.pad(img, ((ph, ph), (pw, pw)), mode="reflect")
    return conv2_valid(imgp, kernel)

# -----
# 4) Konvolusi (SAME)
# -----
denoised = conv2_same(noisy, H_norm, pad_mode="reflect")

# -----
# 5) Metrik MSE & PSNR (opsional)
# -----
def mse(a, b):
    return float(np.mean((a - b) ** 2))

def psnr(a, b, data_range=1.0):

```



```

    m = mse(a, b)
    if m == 0: return float('inf')
    return 10.0 * np.log10((data_range ** 2) / m)

print("MSE asli vs asli :", mse(gray, gray))
print("PSNR asli vs asli :", psnr(gray, gray), "dB")
print("MSE noisy vs asli :", mse(noisy, gray))
print("PSNR noisy vs asli :", psnr(noisy, gray), "dB")
print("MSE hasil vs asli :", mse(denoised, gray))
print("PSNR hasil vs asli :", psnr(denoised, gray), "dB")

# -----
# 6) Visualisasi
# -----
plt.figure(figsize=(12,4))
plt.subplot(1,3,1); plt.imshow(gray, cmap="gray"); plt.title("Lena (grayscale)");
plt.subplot(1,3,2); plt.imshow(noisy, cmap="gray"); plt.title(f"Noisy ( $\sigma$ = $\{\sigma\}$ )")
plt.subplot(1,3,3); plt.imshow(denoised, cmap="gray"); plt.title("Conv2 SAME (H/12,")
plt.tight_layout(); plt.show()

```

```

In [ ]: import numpy as np
        from PIL import Image
        import matplotlib.pyplot as plt

# -----
# 1) Load & grayscale 0..1
# -----
img_rgb = Image.open("91.png").convert("RGB")
arr = np.asarray(img_rgb).astype(np.float32) / 255.0
R, G, B = arr[...,0], arr[...,1], arr[...,2]
gray = 0.299*R + 0.587*G + 0.114*B # Luminance BT.601

# -----
# 2) Tambahkan noise Gaussian
# -----
sigma = 0.1 # atur besaran noise (0.02 s.d. 0.1 umumnya cukup terlihat)
rng = np.random.default_rng(42)
noisy = np.clip(gray + rng.normal(0.0, sigma, size=gray.shape), 0.0, 1.0)

# -----
# 3) Kernel H dan conv2_same
# -----
H = np.array([[1, 2, 1],
               [2, 4, 2],
               [1, 2, 1]], dtype=np.float32)

H_norm = H / H.sum() # normalisasi agar intensitas terjaga (jumlah=1) 1/12 * jumlah

def conv2_valid(img, kernel):
    kh, kw = kernel.shape
    ih, iw = img.shape
    kflip = kernel[::-1, ::-1]
    oh, ow = ih - kh + 1, iw - kw + 1
    out = np.zeros((oh, ow), dtype=np.float32)
    for i in range(oh):
        for j in range(ow):

```

```

        window = img[i:i+kh, j:j+kw]
        out[i, j] = np.sum(window * kflip)
    return out

def conv2_same(img, kernel, pad_mode="reflect"):
    kh, kw = kernel.shape
    ph, pw = kh // 2, kw // 2
    if pad_mode == "replicate":
        imgp = np.pad(img, ((ph, ph), (pw, pw)), mode="edge")
    elif pad_mode == "constant":
        imgp = np.pad(img, ((ph, ph), (pw, pw)), mode="constant", constant_values=0)
    else: # default: reflect (recommended for tepi)
        imgp = np.pad(img, ((ph, ph), (pw, pw)), mode="reflect")
    return conv2_valid(imgp, kernel)

# -----
# 4) Konvolusi (SAME)
# -----
denoised = conv2_same(noisy, H_norm, pad_mode="reflect")

# -----
# 5) Metrik MSE & PSNR (optional)
# -----
def mse(a, b):
    return float(np.mean((a - b) ** 2))

def psnr(a, b, data_range=1.0):
    m = mse(a, b)
    if m == 0: return float('inf')
    return 10.0 * np.log10((data_range ** 2) / m)

print("MSE asli vs asli :", mse(gray, gray))
print("PSNR asli vs asli :", psnr(gray, gray), "dB")
print("MSE noisy vs asli :", mse(noisy, gray))
print("PSNR noisy vs asli :", psnr(noisy, gray), "dB")
print("MSE hasil vs asli :", mse(denoised, gray))
print("PSNR hasil vs asli :", psnr(denoised, gray), "dB")

# -----
# 6) Visualisasi
# -----
plt.figure(figsize=(12,4))
plt.subplot(1,3,1); plt.imshow(gray, cmap="gray"); plt.title("Lena (grayscale)");
plt.subplot(1,3,2); plt.imshow(noisy, cmap="gray"); plt.title(f"Noisy ( $\sigma=\{\text{sigma}\}$ )");
plt.subplot(1,3,3); plt.imshow(denoised, cmap="gray"); plt.title("Conv2 SAME (H/12,");
plt.tight_layout(); plt.show()

```

```

In [ ]: import numpy as np
        from PIL import Image
        import matplotlib.pyplot as plt

        # -----
        # 1) Load & grayscale 0..1
        # -----
        img_rgb = Image.open("01.png").convert("RGB")
        arr = np.asarray(img_rgb).astype(np.float32) / 255.0

```

```

R, G, B = arr[...,0], arr[...,1], arr[...,2]
gray = 0.299*R + 0.587*G + 0.114*B # Luminance BT.601

# -----
# 2) Tambahkan noise Gaussian
# -----
sigma = 0.1 # atur besaran noise (0.02 s.d. 0.1 umumnya cukup terlihat)
rng = np.random.default_rng(42)
noisy = np.clip(gray + rng.normal(0.0, sigma, size=gray.shape), 0.0, 1.0)

# -----
# 3) Kernel H dan conv2_same
# -----
H_norm = np.array([[0, -1, 0],
                   [-1, 5, -1],
                   [0, -1, 0]], dtype=np.float32)

# H_norm = H / H.sum() # normalisasi agar intensitas terjaga (jumlah=1) 1/12 * jum

def conv2_valid(img, kernel):
    kh, kw = kernel.shape
    ih, iw = img.shape
    kflip = kernel[::-1, ::-1]
    oh, ow = ih - kh + 1, iw - kw + 1
    out = np.zeros((oh, ow), dtype=np.float32)
    for i in range(oh):
        for j in range(ow):
            window = img[i:i+kh, j:j+kw]
            out[i, j] = np.sum(window * kflip)
    return out

def conv2_same(img, kernel, pad_mode="reflect"):
    kh, kw = kernel.shape
    ph, pw = kh // 2, kw // 2
    if pad_mode == "replicate":
        imgp = np.pad(img, ((ph, ph), (pw, pw)), mode="edge")
    elif pad_mode == "constant":
        imgp = np.pad(img, ((ph, ph), (pw, pw)), mode="constant", constant_values=0)
    else: # default: reflect (recommended for tepi)
        imgp = np.pad(img, ((ph, ph), (pw, pw)), mode="reflect")
    return conv2_valid(imgp, kernel)

# -----
# 4) Konvolusi (SAME)
# -----
denoised = conv2_same(noisy, H_norm, pad_mode="reflect")

# -----
# 5) Metrik MSE & PSNR (opsional)
# -----
def mse(a, b):
    return float(np.mean((a - b) ** 2))

def psnr(a, b, data_range=1.0):
    m = mse(a, b)
    if m == 0: return float('inf')

```

```
    return 10.0 * np.log10((data_range ** 2) / m)

print("MSE asli vs asli :", mse(gray, gray))
print("PSNR asli vs asli :", psnr(gray, gray), "dB")
print("MSE noisy vs asli :", mse(noisy, gray))
print("PSNR noisy vs asli :", psnr(noisy, gray), "dB")
print("MSE hasil vs asli :", mse(denoised, gray))
print("PSNR hasil vs asli :", psnr(denoised, gray), "dB")

# -----
# 6) Visualisasi
# -----
plt.figure(figsize=(12,4))
plt.subplot(1,3,1); plt.imshow(gray, cmap="gray"); plt.title("Lena (grayscale)");
plt.subplot(1,3,2); plt.imshow(noisy, cmap="gray"); plt.title(f"Noisy ( $\sigma$ ={{sigma}})")
plt.subplot(1,3,3); plt.imshow(denoised, cmap="gray"); plt.title("Conv2 SAME (H/12,
plt.tight_layout(); plt.show()
```

In []:

In []: